SYSTEM DESIGN

Query Optimization Visual Guide

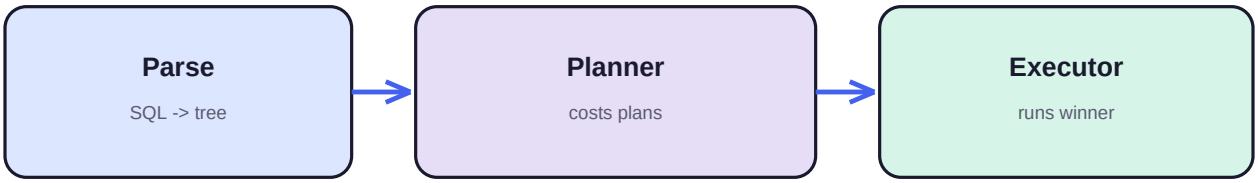
From SQL text to fast execution:
planners, joins, statistics, and traps.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Planner vs Executor

You write WHAT; the planner picks HOW; the executor runs it.



- The planner enumerates strategies (join orders, scan types) and COSTS each using table statistics.
- Cheapest estimated plan wins -- the planner is only as good as its statistics.
- Cost is in abstract units (page reads + CPU), not seconds: compare plans relatively, then measure.

STEP 2/6

Reading EXPLAIN

The plan is a tree: leaves scan, nodes join, root returns.

```
Hash Join  (cost=.... rows=100)
  ->  Seq Scan on orders
  ->  Hash  ->  Index Scan on users
```

- Read bottom-up: scans feed joins feed sorts. Indentation = data flow.
- cost=startup..total: startup matters for LIMIT queries (cheap start wins).
- rows: the planner's estimate. EXPLAIN ANALYZE shows actual rows + real milliseconds -- trust actuals.
- Huge estimate error (100 vs 1M) = stale statistics: ANALYZE the table.

STEP 3/6

Join Strategies

Three algorithms; the planner picks by size and indexes.

Nested loop

for each outer row, probe inner -- wins when outer is TINY and inner is indexed

Hash join

build hash table of smaller side, probe with larger -- the big-table workhorse

Merge join

both sides sorted (or index-ordered) -- streams, tiny memory

A nested loop over 1M x 1M rows is a trillion probes -- if you see it on big tables, add the missing index.

STEP 4/6

Statistics Drive Everything

ANALYZE samples the table: row counts, distinct values, histograms, correlations.

planner believes: status='active' matches 90% of rows

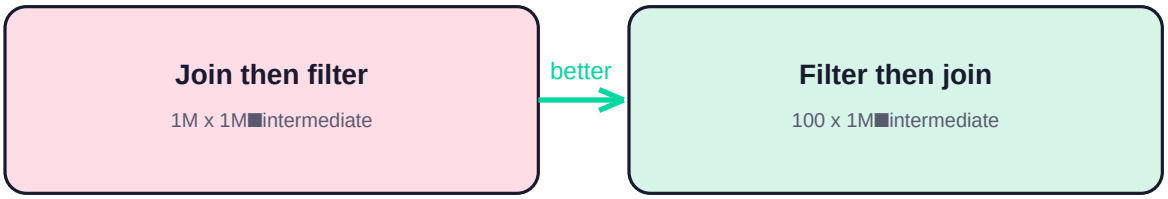
reality after a purge: it matches 2% of rows

- Stale stats make the planner choose seq scans for selective filters and nested loops for huge joins.
- Autovacuum usually keeps up; after bulk loads/deletes, ANALYZE manually.
- Histograms matter for skewed columns (status, country): default stats assume uniformity.

STEP 5/6

Selectivity: Filter Early

Apply the most selective predicate first -- shrink rows before joining.



- Push predicates down: WHERE on one table should filter BEFORE the join, not after.
- The planner usually does this -- unless views, CTEs (materialized), or functions block it.
- Covering indexes on the filter columns turn 'filter' into 'index-only seek'.

STEP 6/6

Common Traps

Queries that LOOK fine and run terribly.

N+1

1 query for users + N queries for their orders -- batch with a join or IN (...)

Functions on columns

WHERE YEAR(created_at)=2024 kills the index -- use a range predicate

Implicit casts

WHERE phone = 123 on a text column -- cast blocks the index

SELECT *

drags LOBs across the wire, defeats covering indexes -- name columns

STEP ✓/✓

Key Takeaways

Query Optimization — Remember

- The planner costs candidate plans using statistics; the executor runs the cheapest estimate.
- Read EXPLAIN bottom-up; EXPLAIN ANALYZE reveals estimate-vs-actual lies.
- Nested loop (tiny+indexed), hash join (big tables), merge join (sorted streams) -- match to shape.
- Stale statistics are the #1 cause of bad plans: ANALYZE after bulk changes.
- Filter early, avoid N+1, never wrap indexed columns in functions, and name your columns.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev